

Internship Task Report: Task 5 Day5

Date: [7.06.2026]

Intern Name: [Mahek.oza]

(Interns Batch):- Cyber security

Target Website: https://kryosync.netlify.app

OWASP Top 10 Overview and Notes:-

The Open Web Application Security Project (OWASP) Top 10 is a globally recognized awareness document representing the most critical security risks to web applications. It serves as a foundational checklist for developers, QA engineers, and security professionals to minimize vulnerabilities during software development.

The OWASP Top 10 Vulnerabilities:-

A01: Broken Access Control

What it is: Access control ensures users cannot act outside of their intended permissions. A failure here allows unauthorized users to view, modify, or delete sensitive data, or even escalate their privileges to administrators.

Common Examples: * IDOR (Insecure Direct Object Reference): Changing an ID parameter in a URL (e.g., app/user?id=1001 to id=1002) to view another user's private data.

Bypassing access checks by modifying metadata or tampering with JSON Web Tokens (JWT).

Prevention:

Adopt a Deny by Default philosophy for all resources.

Implement centralized access control mechanisms and reuse them across the entire application.

Ensure unique identifiers are cryptographically secure or checked rigorously against current user session permissions.

A02: Security Misconfiguration

What it is: This occurs when application stacks are poorly configured, leaving openings for attackers. It spans everything from unpatched software and default passwords to overly revealing error messages.

Common Examples:

Keeping default administrative accounts and passwords active.

Leaving unnecessary features, services, or ports enabled.

Displaying verbose stack traces or detailed error messages to end-users (which leak database types, versions, or system directories).

Prevention:

Harden environments by disabling unnecessary features, components, and documentation.

Automate the deployment process to ensure configurations are consistently built and verified across dev, test, and production.

A03: Software Supply Chain Failures

What it is: Modern applications heavily rely on third-party frameworks, libraries, and modular plugins. If an attacker compromises an upstream dependency, a malicious update can easily propagate to thousands of applications down the line.

Common Examples:

Utilizing an open-source library that has known CVEs (Common Vulnerabilities and Exposures) or has been injected with a malicious payload by a rogue contributor.

Lack of code signing or validation on updates downloaded by the software.

Prevention:

Maintain an up-to-date Software Bill of Materials (SBOM) to track all application dependencies.

Use Software Composition Analysis (SCA) tools to scan codebases for vulnerable third-party components continuously.

A04: Cryptographic Failures

What it is: Formerly known as Sensitive Data Exposure, this focuses on failures related to cryptography (or lack thereof), which directly leads to data exposure or system compromise.

Common Examples:

Transmitting data in clear text via unencrypted HTTP protocols.

Relying on old or broken cryptographic algorithms (e.g., MD5, SHA-1).

Storing user passwords using plain text or weak hashing strategies rather than strong, modern functions.

Prevention:

Enforce data encryption in transit using secure protocols like TLS and HTTP Strict Transport Security (HSTS).

Encrypt sensitive data at rest using robust algorithms (e.g., AES-256).

Hash passwords using salted, adaptive hashing functions like bcrypt, scrypt, or Argon2.

A05: Injection

What it is: Injection occurs when an application passes untrusted data to an interpreter as part of a command or query. The attacker's hostile data tricks the interpreter into executing unintended commands or accessing data without authorization.

Common Examples:

SQL Injection (SQLi): An attacker enters structured commands into a login box to bypass verification and query database contents directly.

Command Injection, LDAP Injection, and Cross-Site Scripting (XSS).

Prevention:

Use parameterized queries (Prepared Statements) to strictly isolate user input from code logic.

Enforce rigorous server-side input validation, sanitization, and context-aware output encoding.

A06: Insecure Design

What it is: This refers to flaws that originate before any code is ever written. Unlike implementation bugs (where a feature is coded incorrectly), insecure design means a feature was designed without essential security controls from the ground up.

Common Examples:

Creating a password reset flow that allows users to answer easily guessable or public security questions without a multi-factor token check.

Designing a ticketing system that doesn't limit the number of tickets a single user can purchase simultaneously, enabling scalper bots.

Prevention:

Shift security left by integrating threat modeling and secure design principles early into the Software Development Life Cycle (SDLC).

Build a library of pre-approved, reusable secure design patterns.

A07: Authentication Failures

What it is: Vulnerabilities associated with establishing and maintaining a user's identity. Weaknesses here make it trivial for attackers to compromise sessions or hijack valid user accounts.

Common Examples:

Permitting weak or easily guessable passwords (e.g., Password123).

Susceptibility to automated credential stuffing or brute-force attacks due to a lack of rate limiting.

Insecure session identifier management (e.g., exposing session IDs in URLs or failing to invalidate them upon logout).

Prevention:

Enforce multi-factor authentication (MFA) to provide defense-in-depth.

Implement strict password complexity requirements and check inputs against lists of known breached passwords.

Utilize built-in, secure session managers provided by trusted modern frameworks rather than creating custom solutions.

A08: Software and Data Integrity Failures

What it is: This relates to code and infrastructure that fail to protect against integrity violations. This happens when applications accept plugins, modules, or serialized data streams from untrusted sources without verification.

Common Examples:

Insecure Deserialization: The application accepts data object states from untrusted users and deserializes them back into memory, which an attacker can exploit to execute arbitrary code.

Auto-updating client-side applications without validating digital signatures.

Prevention:

Use digital signatures, checksums, or similar verification mechanisms to ensure code or data hasn't been altered during transit/storage.

Avoid accepting serialized objects from untrusted sources; pass data through generic, safe formats like JSON or XML instead.

A09: Security Logging and Alerting Failures

What it is: Without proper logging, visibility into a system is completely lost. Security breaches can go completely undetected for months if log events are missing, unmonitored, or stored insecurely.

Common Examples:

Auditable security events (such as logins, failed login attempts, or access control rejections) are not recorded.

Logs are stored locally and can be modified or cleared by an attacker who gains minimal server access.

Prevention:

Ensure all login, access control, and server-side validation failures are comprehensively logged with sufficient context.

Centralize log management so logs can be monitored and analyzed securely in real time.

Establish automated alerts for suspicious patterns (e.g., multiple failed privilege attempts within a short window).

A10: Mishandling of Exceptional Conditions

What it is: This flaw revolves around how an application reacts when encountering unexpected errors, runtime panics, or system edge cases. Poor error handling can lead to unexpected state transitions, data leakage, or service denial.

Common Examples:

An unhandled null pointer or out-of-bounds error crashes a backend thread, leading to a resource exhaustion Denial of Service (DoS).

An unexpected application exception short-circuits an authorization check block, failing “open” and inadvertently letting the request bypass security checks.

Prevention:

Enforce robust error handling policies throughout the codebase (try-catch structures with strict fallbacks).

Ensure that systems fail securely (fail-closed) whenever an exception occurs, rather than skipping logic blocks or leaving connections open.

Website security header:-

1. Content-Security-Policy (CSP)

Status: Missing

Risk: High (Susceptible to Cross-Site Scripting (XSS), data injection, and malicious script execution).

Impact: Without a CSP, the browser will trust and execute any scripts loaded by the application, whether they come from your code, third-party CDNs, or an injected malicious source.

2. Strict-Transport-Security (HSTS)

Status: Warning / Partial (Netlify enforces HTTPS globally by default, but explicit HSTS headers with long durations are ideal).

Risk: Low.

Impact: Enforcing a strict max-age prevents protocol downgrade attacks and ensures that modern browsers only load the site securely over HTTPS.

3. X-Frame-Options

Status: Missing

Risk: Medium (Susceptible to Clickjacking).

Impact: If missing, a malicious actor can overlay your website inside an invisible <iframe> on their own domain, tricking users into clicking buttons they did not intend to interact with.

4. X-Content-Type-Options

Status: Missing

Risk: Medium (MIME-Sniffing exploitation).

Impact: Missing this directive allows browsers to guess the content type of files instead of following the official Content-Type header, which can cause them to execute text/plain assets as active JavaScript.

5. Referrer-Policy & Permissions-Policy

Status: Missing

Risk: Low (Information Leakage / API Abuse).

Impact: Lacking a referrer policy can leak precise internal routing URLs to external third-party sites when user navigation actions occur. Similarly, a missing Permissions Policy allows embedded components access to native hardware features (like geolocation or microphone).

Remediation Plan (Netlify Implementation)

To fix these missing headers on a Netlify app, create a file named `_headers` inside your project's main build distribution directory (usually the root folder or the `public/` directory depending on your framework).

Add the following configuration blocks to secure the site: